

ECE 1387 - CAD for Digital Circuit Synthesis and Layout

**Exercise #1 - Simulated Annealing-based Placement and
Timing-Driven Negotiated Congestion Routing**

October 2025

J. Anderson

Assignment Date: October 30, 2025
Due Date: November 7, 2025, 23:59 Toronto time, with PDF report by email to Jason
Late Penalty: -1 mark per day late, with total marks available = 10

The purpose of this exercise is two-fold: 1) to use an automatic placement tool based on the simulated annealing optimization strategy, and to gain some familiarity with the properties of that strategy; and, 2) to experiment with the negotiated congestion routing algorithm described in class.

The placement and routing (P&R) tool you will use in this exercise is called “VPR”. It is a tool originally created here at UofT by Prof. Vaughn Betz as part of his Ph.D research in the late 90s. It has been enhanced extensively since then by several other graduate students, and is now at version 9. As well as being a P&R tool, VPR is a framework for FPGA CAD and architectural research, used by researchers all around the world. VPR takes a textual description of the target FPGA architecture as input. The description specifies, among other things, the make-up of the FPGA logic blocks, the routing segment lengths, the switch block style, and the routing delays. By changing the architecture description file, one can evaluate the speed and area efficiency of different FPGAs.

VPR is part of a larger FPGA CAD toolsuite called VTR (Verilog-to-Routing), which, besides placement and routing with VTR, includes RTL synthesis, logic optimization and technology mapping.

Access to Software

Download VTR from the *course* website link and copy it into your ECF account. Go to the top-level directory of the VTR distribution, and type: make vpr. A binary executable, vpr/vpr will be produced. I have verified this works on the ECF machines.

Instruction Manual for VPR

Documentation is available online: <https://docs.verilogtorouting.org/en/latest/vpr/>

Netlist Files and Architecture File to Use

Use the architecture file available on the website: k6_N10_mem32K_40nm.xml. You may find reading the architecture file interesting, as it gives a description of an FPGA architecture. Take a look at the top of the file. The architecture file describes an FPGA with ten 6-input LUTs per logic block, columns of RAM and columns of multipliers. The routing segments span 4 logic-block tiles.

The benchmarks to use for this exercise are also on the website (see four files with .blif extension). Unlike in the course assignments so far, which used synthetic circuits, these are *real* circuits. BLIF refers to Berkeley Logic Interchange Format – a circuit netlist format.

Exercise A - Placement and Simulated Annealing

The VPR program allows the user to set various Simulated Annealing parameters: the starting temperature, the ending temperature, the rate at which the temperature decreases, the number of moves per temperature, and the initial random seed. *NOTE: For **all** steps in Exercise A, run VPR with the following parameters: "--pack --place" (prevent routing) and "--place_algorithm bounding_box" (minimize HPWL only).*

1. Run the vpr program once, with its default parameters, on four circuit netlists. Plot the score (cost function = HPWL) versus the temperature.
2. Run VPR 5 times for each circuit with different random seeds. Calculate and report the mean and standard deviation of the resulting final scores. Comment on the sensitivity of the scores to the random seeds? Do all circuits exhibit the same sensitivity to the random seed?
3. For each circuit, run VPR 5 times (with different seeds) with 5 different starting temperatures (25 * 4 circuits = 100 runs in total). Be sure to choose some starting temperatures that are low enough to actually impact the placement results (**NOTE:** you may need to use starting temperatures considerably less than 1!). You may also need to set the exit temperature for meaningful results. For each circuit, report the mean and standard deviation of the final score for each starting temperature.
4. An idea suggested in class that may help improve simulated annealing results is to bump-up the temperature at the end of annealing, and continue the annealing process, in a bid to achieve a lower cost. In vpr/src/place/annealer.cpp, find the outer_loop_update function. Change the function so that it bumps up the temperature to prolong the annealing process. Allow several temperature bumps to occur. Repeat step #2 above, reporting the mean and standard deviation of the placement scores with the temperature bump-up change. Compare the results with those of #2. Explain in your report how you bumped up the temperature.

Exercise B - Timing-Driven Routing

Note: For this part of the Exercise, please use the original VPR and not the one you altered in Part A.

1. VPR implements timing-driven routing using an improved version of the PathFinder approach described in class. Run the placement tool with the default options *and* the timing-driven router. **NOTE:** Do not set "--pack --place" or "--place_algorithm" for Exercise B. Also for Exercise B, set the #tracks/channel (W) to 42 using "--route_chan_width 42" for *sha*, set W to 48 for *reygentop*, set W to 72 for *or1200*, and set W to 56 for *boundtop*. Repeat this 5 times using 5 different placer seeds. Report the mean and standard deviation of post-routing critical path delay (see VPR output).
2. Using the same parameters as in the previous step, explore the different parameters of the router (as described in the documentation) and try to improve the critical path delay for the four circuits. Comment on the parameters you experimented with and their effect on the critical path delay. Be sure to vary the "present congestion" and "historical congestion" parameters talked about in class.
3. *Impact of net and load ordering on critical path delay.* VPR uses a particular ordering when it routes nets, and the loads on nets. Reverse the order that the router routes nets and load pins. This can be done by altering the code at around line 205 of vpr/src/route/route_net.tpp and around line 24 of vpr/src/route/SerialNetlistRouter.tpp. Repeat Step #1 of Exercise B. Comment on the differences in the achieved critical path delays.